

Transaction Risk Review Assistant

A Java + AI portfolio prototype for evidence-led transaction review.

Computer Engineering, Mae Fah Luang University

RiskDesk

TRANSACTION REVIEW

PT

Portfolio workspace

Phyo Thant Kyaw

WORKSPACE

Review queue

3

Rule policy

Automation

Import CSV

Portfolio prototype

Evidence-led review.

Human judgment stays in control.

View PT's portfolio

→

Workspace / Transaction review

SYNTHETIC DATA

Reviewer connected

PT

RISKDESK / REVIEW OPERATIONS

Transaction review

Refresh data

Inspect the signals. Follow the evidence. Record your decision.

Accounts

3

Fictional accounts

High priority

1

Score 50 or above

Awaiting review

2

Pending a decision

Reviewed

1

Decision recorded

Account queue

3

Search name or account ID

All

High

Medium

Low

01

Everyday account

DEMO-001

→

Low Priority

0/100

Pending review

02

Small business account

DEMO-002

→

Medium Priority

35/100

Explained

03

Reactivated account

DEMO-003

→

High Priority

80/100

Pending review

Account review snapshot

05 Oct. 19:00 - Bangkok time

DEMO-003 / THE ACCOUNT

Reactivated account

80

review score / 100

After a long activity gap, an incoming deposit is quickly distributed to five new recipients. Fictional activity designed to exercise multiple rules.

High Priority

Transfer burst

24h incoming

฿100,000

24h outgoing

฿90,000

30-day median outgoing

Insufficient history

0 baseline transactions

Rule findings 4

Transactions

AI analysis

Review history

Evidence-grounded analysis

Rule-based template

4 configured rules triggered: Transfer burst, Large transfer to new recipient, Rapid movement of incoming funds, Activity after a long gap. Review the linked transactions and the customer's legitimate context before taking further action.

Suggested reviewer checks

Inspect the transaction evidence for each triggered rule.

Confirm the payment purpose and relationship to recipients.

Record the review rationale; escalate only if further evidence supports it.

This is a synthetic-data review assistant. Rules and AI explanations do not establish fraud or account safety.

Generate rule explanation

Reviewer decision

Record checks and next steps.

Review status

Pending review

Review note

What did you verify? What needs follow-up?

Reviewer access supplied

Access

Save review

The score prioritizes review. A human reviewer makes the decision.

Built by Phyo Thant Kyaw · Java / TypeScript / SQL / AI

Portfolio prototype · Synthetic data only

Live demo: transaction-risk-review.onrender.com
Source: github.com/pt22-mfu/transaction-risk-review

Project overview

What the system does, what is running, and what its outputs mean.

Problem and approach

Unusual transaction activity needs prioritization and context. RiskDesk applies five explicit Java rules to fictional account histories, links each signal to evidence, optionally explains the findings with Gemini, and lets a person record a review decision.

Capability	Status as of 06 October 2026
Public web application	Deployed on Render; updated teal/gray UI confirmed by the owner.
Database	Supabase PostgreSQL configured; cloud account reads, CSV imports and saved review history demonstrated. Controlled restart persistence has not been separately tested.
Gemini explanations	Configured and demonstrated on a seeded and an imported account.
Automated checks	39 Java tests in the recorded MVP verification; strict frontend build and redesigned UI browser checks passed.
n8n automation	Workflow export and protected report endpoint exist. Digest transformation tested; hosted schedule and delivery not activated.

Who this documentation is for

A developer maintaining the project, a reviewer trying the demo, and an interviewer evaluating the engineering choices. The handbook explains the current implementation rather than a planned production platform.

Interpretation: the review score is a handcrafted priority score, not a probability of fraud. A high score requests human checks; a score of zero does not establish account safety.

Core boundaries

Fictional data only; THB amounts; no real bank connection, balances, KYC, watchlists, account freezing, money movement or automatic enforcement. Current reviewer access is a shared demo token, not an individual identity system.

Reader guide

Use this as an operating manual and a quick implementation reference.

Section	Page	Section	Page
System architecture	4	API examples	15
Database and persistence	5	Frontend user guide	16
Time windows and amount semantics	6	Configuration reference	17
Rules: burst, baseline and recipients	7	Local setup and development	18
Rules: movement, inactivity and scoring	8	Cloud deployment and updates	19
Synthetic scenarios and expected results	9	Verification and acceptance	20
CSV input contract	10	Troubleshooting	21
Import processing and integrity	11	n8n daily digest	22
AI explanations and fallback	12	Limitations and future improvements	23
Review workflow and access	13	Demo, interview and source guide	24
REST API reference	14		

The shortest mental model

CSV or seeded history → Java rules → score + evidence → optional AI explanation → human review → stored audit history.

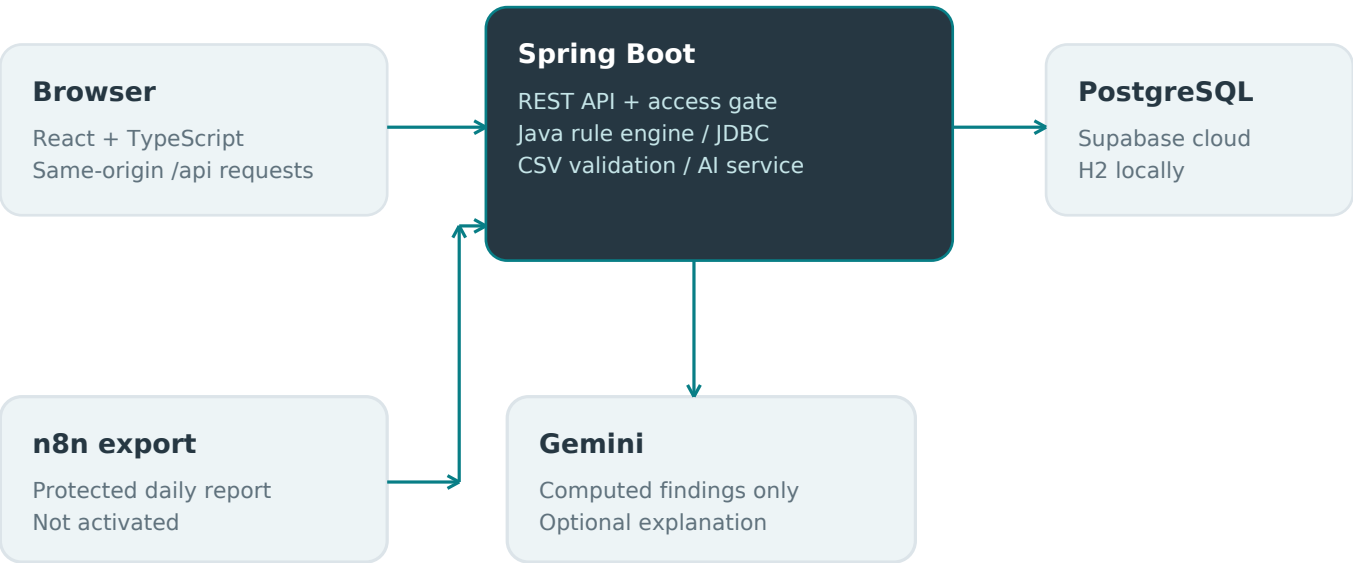
When you forget something

- Need to explain a score? Start with pages 6-9.
- Need to change data or API behavior? Use pages 5, 10-15.
- Need to run, deploy or diagnose the app? Use pages 17-21.
- Need an interview walkthrough? Use page 24.

All times in examples are timezone-aware. The UI displays Bangkok time; stored/imported event timestamps are handled as instants. Credentials are represented by placeholders throughout this document.

System architecture

One deployed application URL, with the backend serving both API and built UI.



Module / location	Responsibility
frontend/src/App.tsx	Account selection, evidence tabs, AI request state, review form and CSV upload.
frontend/src/styles.css	Teal/gray design, desktop review columns and responsive layouts.
ApiController / ApiErrors	REST routes, input validation, token gate and expected error responses.
RiskEngine / Policy	Deterministic account analysis and configurable thresholds.
RiskRepository	Parameterized JDBC queries, review records and transactional seeding/imports.
CsvImportService	Decode, parse and validate CSV before any database write.
ExplanationService / AiSettings	Provider call, response validation, rate limit and labelled fallback.
DemoData / Models	Seed fixtures and typed Java records for data/API responses.

Request lifecycle: the browser requests an account, the repository retrieves its history and cutoff, the engine analyzes it, and the API returns transactions, findings and reviews. Provider calls are made by java on the server; the browser does not receive database credentials or a Gemini key.

Database and persistence

Four tables in the private risk_demo schema.

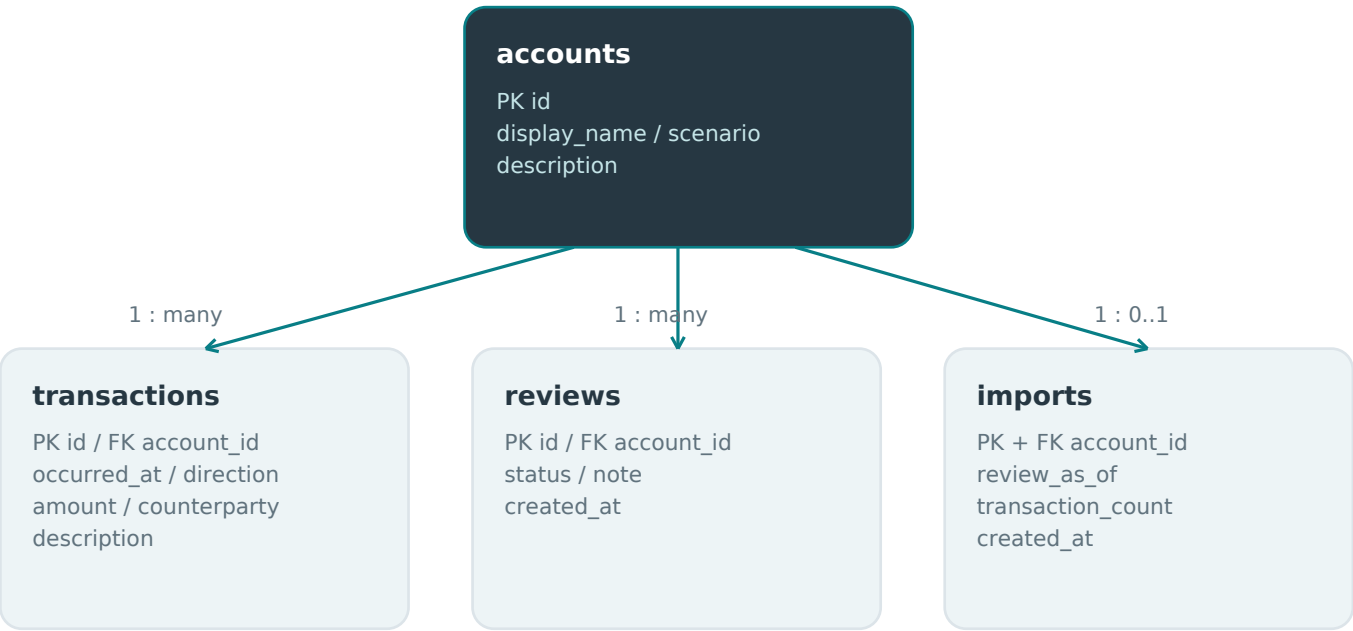


Table	Important fields and constraints
accounts	id VARCHAR(40) primary key; display_name VARCHAR(120); scenario VARCHAR(80); description VARCHAR(1000). All required.
transactions	id VARCHAR(60) primary key; account_id foreign key; occurred_at TIMESTAMP WITH TIME ZONE; direction IN/OUT; amount NUMERIC(18,2) > 0; counterparty VARCHAR(80); description VARCHAR(300).
reviews	id VARCHAR(40) primary key; account_id foreign key; status restricted to three values; note VARCHAR(2000); created_at timezone-aware.
imports	One row per imported account: account_id primary/foreign key, review_as_of, transaction_count and created_at. Seeded accounts have no import row.

Persistence behavior

Indexes support transactions by account/time and reviews by account/time. Review history is read newest first. The latest record supplies the dashboard status. CSV writes insert the account, its transactions and import metadata in one JDBC transaction. Startup seeding runs only when the accounts table is empty.

Supabase setup enables RLS and revokes table access from anon/authenticated roles. The server connects using the database credential; the browser uses the Java API. Keep risk_demo outside exposed Data API schemas. Local H2 uses the compatible schema in src/main/resources/schema.sql.

Time windows and amounts

Event time, not the current wall clock, determines the review.

Earlier supplied history still informs recipient and inactivity rules.



Cutoff and inclusivity

- **Seeded accounts:** asOf = 2026-10-05T12:00:00Z, which displays as 05 October, 19:00 in Bangkok.
- **Imported accounts:** asOf is the latest transaction timestamp in that file, saved in imports.review_as_of.
- **Review window:** windowStart = asOf - 24 hours; both exact windowStart and exact asOf are included. Later events and other accounts are excluded.
- **Amount baseline:** outgoing events from [windowStart - 30 days, windowStart). At least five are needed by the amount rule.
- **Recipient and inactivity history:** all supplied events before windowStart are available; these rules are not restricted to the 30-day amount baseline.

Ordering and date handling

Transactions are sorted by occurredAt and then ID. Imports require an ISO timestamp with an offset and are normalized to UTC. Equal timestamps cannot establish that an outgoing payment occurred after a deposit, so they are excluded from rapid-movement evidence.

Decimal arithmetic

Java uses BigDecimal and the database stores NUMERIC(18,2). The median sorts outgoing amounts; with an even count it averages the two middle amounts and rounds to two decimals using HALF_UP. Do not use binary floating point for these amount comparisons.

UI detail: the current money formatter displays whole THB amounts. Backend calculations retain decimal precision; values such as 10000.50 may appear rounded in the UI. A future release can display two decimal places consistently.

Rules: burst, amount and recipients

Thresholds are configuration values; rule points are constants in RiskEngine.

BURST / Transfer burst / +25

Trigger when at least five outgoing payments occur in an elapsed ten-minute interval inside the 24-hour review window. The engine starts a sliding interval at each outgoing event and uses the first qualifying burst. The exact ten-minute boundary is included.

Example: payments at 09:02, 09:04, 09:06, 09:08 and 09:10 span eight minutes and trigger. Payments need not share the same clock-minute bucket. A planned supplier batch is a possible legitimate explanation.

AMOUNT / Amount above baseline / +20

Requires at least five historical outgoing payments in the 30-day baseline. A review-window outgoing payment must be at least THB 10,000 and strictly greater than five times the historical median.

```
baseline_count >= 5  
amount >= 10000 AND amount > (median * 5)
```

Boundary: if median = 2,000, exactly 10,000 does not trigger because it equals 5x. A payment of 10,001 does. No sufficient history means the rule is skipped, not that an amount is normal.

RECIPIENT / Large transfer to new recipient / +15

Build a set of counterparties from all outgoing history before windowStart. A window outgoing payment of at least THB 10,000 triggers if its counterparty is not in that set. Counterparty strings are matched exactly after import trimming; aliases are not resolved.

Important: recipients are compared with pre-window history. A recipient first seen within the same review window remains new under this definition. Missing history or a genuine new supplier can produce the same signal.

Each rule adds its points once per account analysis, even if several transactions satisfy it. Evidence transaction IDs identify which supplied records support the finding.

Rules: movement, inactivity and score

A temporal signal is a reason to investigate, not proof of a causal money trail.

RAPID / Rapid movement of incoming funds / +25

Consider an incoming deposit of at least THB 20,000 inside the review window. Sum strictly later outgoing events occurring within 30 minutes, including the exact deadline, and stop at the next incoming event. Trigger if the sum is at least 80% of the deposit.

Example: a THB 100,000 deposit followed by THB 90,000 outgoing within ten minutes triggers: $90,000 / 100,000 = 0.90$. A payment before the deposit does not count; a payment at the identical timestamp does not prove an ordering. The next incoming event stops collection even if it is small.

DORMANT / Activity after a long gap / +15

Requires some supplied pre-window history, activity in the review window and at least one outgoing payment in that window. Measure the gap between the last pre-window event and the first window event. A gap of at least 30 days triggers.

The gap is measured at resumption, not from the previous event to the analysis cutoff. With no earlier history, inactivity cannot be established. Evidence includes the previous event, first resumed event and first outgoing event, with duplicate IDs removed.

Score and priority

```
score = min(100, sum(points for triggered rules))
HIGH:  50-100
MEDIUM: 20-49
LOW:    0-19
```

Finding combination	Score	Meaning
AMOUNT + RECIPIENT	20 + 15 = 35	Medium priority: investigate a large new-supplier payment.
BURST + RECIPIENT + RAPID + DORMANT	25 + 15 + 25 + 15 = 80	High priority: multiple signals need context.

Rules can overlap. The score is not statistically calibrated and is not a fraud classifier. Threshold changes belong in application.yml / Policy; changing the point weights requires editing RiskEngine and updating its tests.

Synthetic scenarios

Small, reproducible fixtures make rule behavior inspectable.

Account	Expected score / rules	Evidence and teaching purpose
DEMO-001 Everyday account	0 / none	11 transactions. Eight historical grocery payments; salary and two ordinary outgoing payments in the window.
DEMO-002 Small business account	35 / AMOUNT + RECIPIENT	10 transactions. Eight historical outgoing amounts of 900-1600; median 1250. A 30,000 payment to SUPPLIER-NEW exceeds 6250 and the minimum amount.
DEMO-003 Reactivated account	80 / BURST + RECIPIENT + RAPID + DORMANT	7 transactions. Earlier activity on 20 August; deposit 100,000 on 05 October; five payments of 18,000 to new recipients.

Why DEMO-003 does not trigger AMOUNT

The old outgoing payment lies outside its 30-day baseline and there are no eligible baseline transactions. The UI correctly shows insufficient history. The other four rules produce the score of 80 without inventing a historical spending baseline.

Evidence walkthrough

Rule	DEMO-003 evidence IDs
BURST	S-102 through S-106 (five outgoing events).
RECIPIENT	S-102 through S-106 (five large new-recipient payments).
RAPID	S-101 plus S-102 through S-106 (deposit and five subsequent payments).
DORMANT	S-H01, S-101 and S-102 (prior activity, resumption and first outgoing).

Import sample

samples/demo-transactions.csv contains seven events: old activity of THB 500, incoming THB 50,000 and five outgoing payments totaling THB 45,000. It also scores 80. Four outgoing payments meet the new-recipient minimum; the fifth is THB 5,000 but still contributes to burst and rapid movement.

These scenarios validate expected logic. They are not labelled real fraud data and do not establish accuracy, precision, recall or fraud-prevention performance.

CSV input contract

One file creates one fictional account; six exact header names are required.

```
transaction_id,occurred_at,direction,amount,counterparty,description
```

Column	Accepted value
transaction_id	1-40 ASCII letters, digits, underscores or hyphens; unique within the file.
occurred_at	ISO timestamp with offset, e.g. 2027-01-04T09:00:00+07:00. Normalized UTC year must be 2000-2100.
direction	IN or OUT. Input is trimmed and converted to uppercase.
amount	Positive decimal, up to 16 integer digits and two decimals. No commas, exponent notation or currency symbol.
counterparty	Trimmed text, 1-80 characters. Use a consistent identifier for the same party.
description	Trimmed text up to 300 characters. May be empty; it is not an instruction to the AI.

File and form requirements

Non-empty UTF-8 CSV, at most 1 MB and 1,000 records. A UTF-8 BOM is accepted. Header names are case-sensitive and must form exactly the required set: no extra or duplicate columns; header order may vary. RFC4180 quoting supports commas and multiline text. Blank lines are ignored.

The form needs an account display name of 1-80 trimmed characters, a file, syntheticConfirmed=true and the reviewer header. All imported values are visible in the shared demo; use invented identifiers and descriptions.

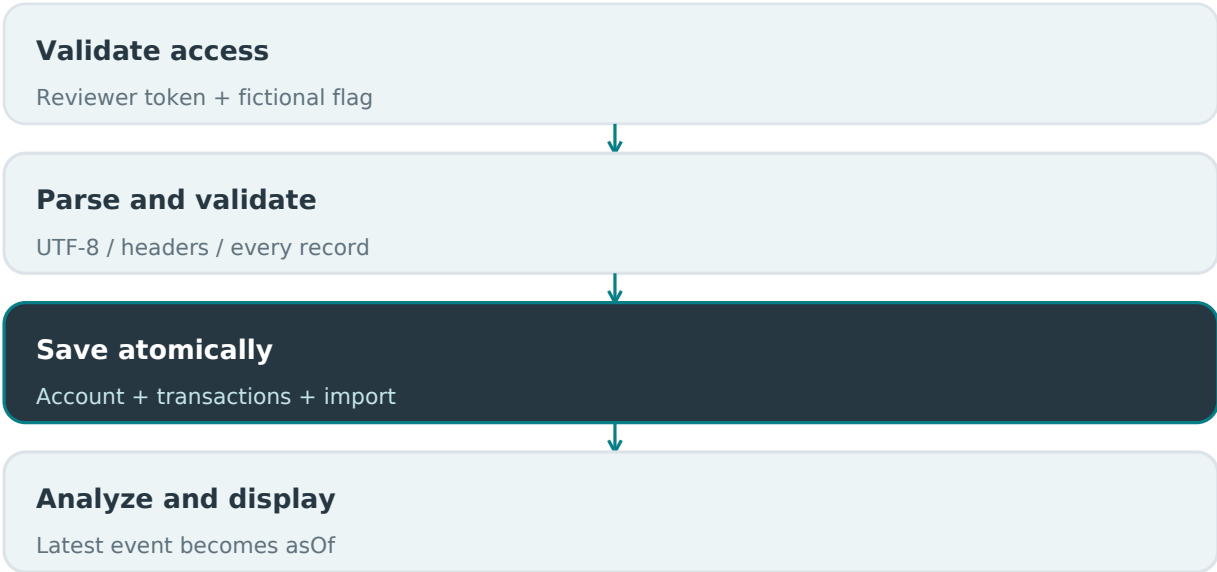
Minimal valid example

```
transaction_id,occurred_at,direction,amount,counterparty,description
T1,2027-01-04T09:00:00+07:00,IN,30000,DEMO-CLIENT,Demo income
T2,2027-01-04T09:10:00+07:00,OUT,25000,DEMO-SUPPLIER,Demo payment
```

This example has no pre-window history. Its timing and recipient signals may trigger, but the amount-baseline and inactivity rules cannot establish the missing history. Use the repository sample for the reproducible score-80 demonstration.

Import processing and integrity

Validation completes before a transaction is opened for database inserts.



Identifiers and snapshots

Generate an account ID of CUSTOM- plus 12 uppercase hexadecimal characters. Persist each transaction ID as accountId + "-" + input transaction_id. This namespaces IDs between imports; the parser still rejects duplicate input IDs within one file. The latest event timestamp becomes that account's review cutoff.

All-or-nothing behavior

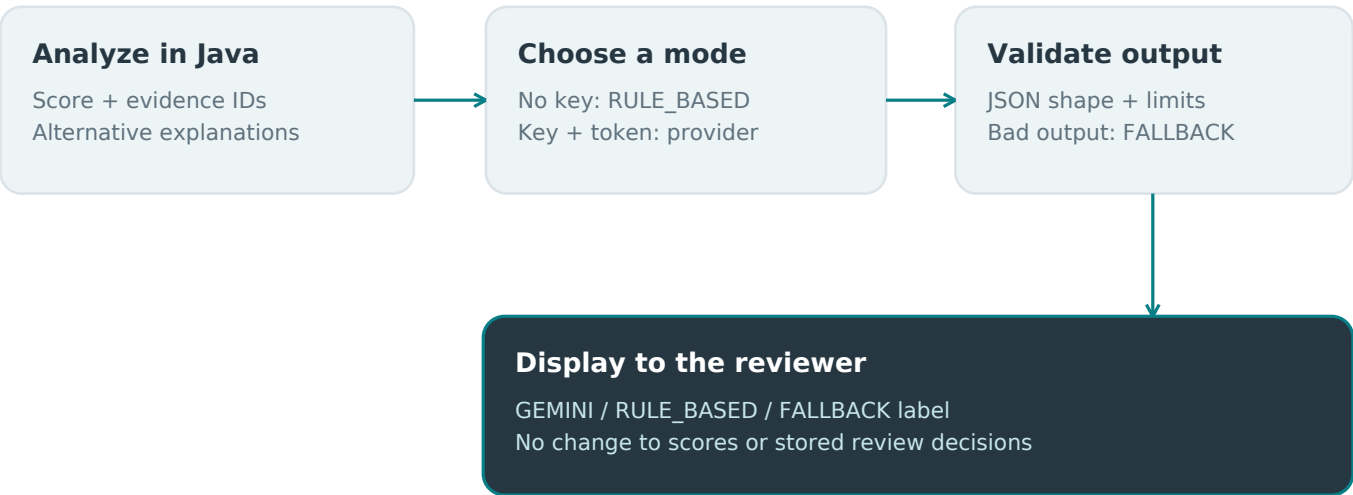
Parsing checks every record and records up to ten detailed row issues. If any validation issue exists, nothing is saved. RiskRepository.importAccount is transactional: the account, all transactions and the import metadata commit together, or a database failure rolls them back.

Condition	Expected result
Invalid headers, amount, date or duplicate ID	400 with error and issues array; no new account.
Oversized multipart upload	413 for the handled upload-size condition.
Missing/incorrect reviewer token	401 when configured; no import.
Reviewer access not configured	503; imports disabled.
Valid complete file	201 with accountId, importedRows and asOf; new account auto-selected in UI.

Re-uploading creates another account. Cross-file deduplication, updates to an existing account, undo/delete and raw-file retention are not implemented. Keep this in mind when preparing a clean public demo.

AI explanations and fallback

The LLM explains computed findings. It does not compute or change the score.



Source value / UI label	Behavior
RULE_BASED / Rule-based template	No server key: return a deterministic explanation. Public request is allowed in this mode.
GEMINI / AI generated	Key configured, token supplied, provider output passes validation.
FALLBACK / Rule fallback	Provider failure, timeout, invalid response, invalid model string or per-process rate limit. The result remains explicitly labelled.

Provider request and validation

POST to /v1beta/models/{model}:generateContent using a server-only x-goog-api-key header. The prompt serializes the Analysis record: score, totals, cutoff, computed explanations and evidence IDs. Raw transaction descriptions, account display names and review notes are not sent.

Generation requests JSON with summary, nextSteps and limitation; temperature 0.1 and maxOutputTokens 1500. Code accepts a non-blank summary of at most 2000 characters and 1-4 textual steps, each at most 600 characters. The requested 150-word summary limit is a prompt instruction, not a strict word-count validator.

The provider must include a textual limitation field, but the returned limitation is replaced by the fixed application statement. Shape validation does not guarantee semantic accuracy; the reviewer must compare the explanation with evidence. No multi-model retry chain is implemented.

Connection timeout: 8 seconds. Request timeout: 25 seconds. Live-call allowance: 6 attempts per minute per Java process; attempted failed calls count. The limiter resets on restart and is not a distributed quota control.

Review workflow and access

A person records the rationale; the system appends a review record.

Reviewer workflow

- Select an account and examine Rule findings and linked Transactions.
- Use AI analysis when helpful; inspect the source label and suggested checks.
- Enter reviewer access using the top-bar control or the Access button.
- Choose a review status, add a fictional note and save.
- Open Review history to see the appended record and timestamp.

Status	Intended interpretation
PENDING_REVIEW	Checks remain open; a saved note may still document progress.
EXPLAINED	The reviewer has recorded a plausible explanation of the supplied activity.
ESCALATED	The reviewer considers further examination necessary.

Validation and audit semantics

Status must match one of the three exact values. Notes must be non-blank and at most 2000 characters; storage trims the note. Each saved review gets a UUID and server timestamp. The API offers no review edit/delete route. "Reviewed" in the dashboard counts latest statuses other than PENDING_REVIEW; saving a pending note does not increase that count.

Shared token boundary

X-Reviewer-Token protects CSV import, review writes, reports and live AI calls. Comparison uses MessageDigest.isEqual. The token stays in page memory and clears on reload. Configuration flags in /api/meta do not expose the token or the Gemini key.

Public read endpoints expose fictional account histories and review notes. A shared token is not login, RBAC, case ownership or actor attribution. The history is append-only through this API, not a tamper-proof audit ledger against database administrators.

REST API reference

JSON responses; reviewer access is supplied through X-Reviewer-Token.

Method / endpoint	Access	Success / purpose
GET /api/health	Public	200: {"status":"ok"}; liveness only.
GET /api/meta	Public	200: snapshot, policy, dataMode, app version and configuration flags.
GET /api/accounts	Public	200: array of account, analysis and latest status.
GET /api/accounts/{id}	Public	200: account, transactions, analysis and reviews.
GET /api/accounts/{id}/analysis	Public	200: freshly computed Analysis record.
POST /api/accounts/{id}/explanation	Token if AI configured	200: Explanation with source, summary, nextSteps and limitation.
POST /api/accounts/{id}/reviews	Token	201: appended Review; JSON status and note.
POST /api/imports	Token	201: imported account; multipart form fields.
GET /api/reports/daily	Token	200: asOf, accounts and scope for n8n.

Expected error responses

Status	When / structure
400	Invalid CSV or review input. CSV: error + issues. Review: generic validation error.
401	Missing or wrong token on a protected operation when reviewer access is configured.
404	Unknown account ID on the documented account routes.
413	Handled upload-size limit exceeded.
503	Reviewer token is not configured for a protected operation.

AI provider failure normally returns a 200 response containing a labelled fallback, not an HTTP 500 or a 429. Expected application errors are handled by ApiErrors; unhandled database or binding failures may use Spring's default error response.

/api/health does not query the database or Gemini. Also check /api/accounts and /api/meta after a deployment. A liveness response alone does not establish readiness of all integrations.

API examples

Copyable PowerShell examples for local inspection and owner-only review.

Read a deterministic analysis

```
$base = "http://localhost:8080"
$a = Invoke-RestMethod "$base/api/accounts/DEMO-003/analysis"
$a | ConvertTo-Json -Depth 10
```

Selected fields for the seeded DEMO-003 response (abridged):

```
{
  "accountId": "DEMO-003",
  "asOf": "2026-10-05T12:00:00Z",
  "windowStart": "2026-10-04T12:00:00Z",
  "riskScore": 80, "priority": "HIGH",
  "historicalOutgoingCount": 0, "historicalMedian": 0,
  "incomingTotal": 100000, "outgoingTotal": 90000,
  "findings": ["BURST", "RECIPIENT", "RAPID", "DORMANT"]
}
```

For brevity, findings above shows rule IDs. The real API returns Finding objects with ruleId, title, points, explanation, transactionIds and alternativeExplanation.

Save a review using your configured token

```
$headers = @{"X-Reviewer-Token" = $env:REVIEWER_TOKEN }
$body = @{"status" = "PENDING_REVIEW"
  "note" = "Fictional evidence checked; verify the deposit source."
} | ConvertTo-Json
Invoke-RestMethod "$base/api/accounts/DEMO-003/reviews" `
  -Method Post -Headers $headers `
  -ContentType "application/json" -Body $body
```

The 201 response contains id, accountId, status, note and createdAt. Set REVIEWER_TOKEN in the terminal to the same private value used by the server; do not insert a hosted secret in this PDF, source code or a public command screenshot.

Request an explanation or report

```
Invoke-RestMethod "$base/api/accounts/DEMO-003/explanation" `
  -Method Post -Headers $headers
Invoke-RestMethod "$base/api/reports/daily" -Headers $headers
```

Frontend user guide

A compact review workspace in teal, cool gray and charcoal.

Main navigation and account queue

Review queue opens the dashboard. Rule policy explains the five thresholds. Automation describes the reporting integration; it does not activate n8n. Import CSV creates a fictional scenario. Search matches account name or ID, and priority filters narrow the queue.

Account tab	What to do
Rule findings	Read the rule and points. Expand the possible legitimate explanation. Use View evidence transactions to inspect only linked records.
Transactions	View full supplied history, or the evidence subset. Show all transactions clears the evidence filter.
AI analysis	Generate an explanation; inspect AI generated, Rule-based template or Rule fallback before interpreting it.
Review history	Read saved status, rationale and server timestamp, newest first.

Interaction behavior

- On larger desktops, reviewer decisions sit beside the evidence. On smaller screens, the form stacks below; narrow mobile layouts use two rows of tabs.
- The header shows account context, priority and a review-score ring. Incoming/outgoing totals cover the 24-hour window, not all history.
- Arrow keys, Home and End navigate the account tabs. Focus outlines remain visible; reduced-motion preference is respected.
- Changing accounts clears the old note and explanation. Stale AI responses are guarded so a prior account's explanation cannot appear on a new account.
- Changing a queue filter does not silently switch an active review. A notice explains when the selected account is outside the current filter.

Before switching accounts or refreshing data, save any note you want to retain. Unsaved notes and generated explanations are held in frontend memory; explanations are not stored in the database.

Access control in the UI

The top bar and review/import forms open the token dialog. The UI may show "Reviewer connected" when a token is supplied; this does not validate the token by itself. The backend validates it when you perform a protected action.

Configuration reference

Environment values override application.yml; do not commit real credentials.

Setting	Default / deployment meaning
PORT	8080 locally; cloud host supplies its listening port.
DB_URL	Local file-based H2 by default. Hosted demo uses a JDBC PostgreSQL Session pooler URL with sslmode=require.
DB_USERNAME / DB_PASSWORD	sa / blank locally. On Supabase use the Connect dialog's database username and password.
DB_INIT_MODE	embedded by default; use never for Supabase after running supabase/setup.sql.
REVIEWER_TOKEN	Blank by default: protected features disabled. Generate a long random secret for hosting.
DEMO_SEED	true; inserts fixtures only when the accounts table is empty.
GEMINI_API_KEY	Blank = deterministic template mode. Keep the key server-side.
GEMINI_MODEL	Source and render.yaml default: gemini-2.5-flash. Hosted demo was configured with gemini-3.5-flash-lite. Model availability depends on the provider account.
risk.as-of	Fixed seeded cutoff in application.yml: 2026-10-05T12:00:00Z. Imports override it per account through metadata.

Supabase JDBC template

```
DB_URL=jdbc:postgresql://<SESSION_POOLER_HOST>:5432/postgres?sslmode=require
DB_USERNAME=postgres.<PROJECT_REF>
DB_PASSWORD=<DATABASE_PASSWORD>
DB_INIT_MODE=never
```

Copy the actual Session pooler values from your project; do not paste the raw postgresql:// URI into DB_URL. The app expects a jdbc:postgresql:// URL and separate credentials. The database password is not a Supabase publishable/API key. Hikari uses at most three connections.

.env.example documents settings, but Spring Boot does not automatically load a .env file here. Set variables in PowerShell or the hosting dashboard. After changing cloud values, allow the service to redeploy.

Local setup and development

Source build on Windows; no Ubuntu or local Docker is required.

Prerequisites and first source build

Use JDK 17+ and Node.js 22+. Check `java -version`, `node --version` and `npm --version`. Maven Wrapper downloads Maven. Keep an older Hadoop Java configuration separate. The GitHub source repo does not include the downloadable bundle's prebuilt run/ JAR.

```
git clone https://github.com/pt22-mfu/transaction-risk-review.git
cd transaction-risk-review
cd frontend
npm ci
npm run build
cd ..
New-Item -ItemType Directory -Force src/main/resources/static | Out-Null
Copy-Item frontend/dist/* src/main/resources/static/ -Recurse -Force
$env:REVIEWER_TOKEN = "local-demo-reviewer-2026"
./mvnw.cmd verify
java -jar target/transaction-risk-review-0.2.0.jar
```

Open `http://localhost:8080`. The example token is for local testing only. Leave `DB_URL` unset to use the default H2 database; leave `GEMINI_API_KEY` unset for deterministic template mode. Stop the app with `Ctrl+C`.

Development loop

Terminal	Command / behavior
Backend, project root	<code>./mvnw.cmd spring-boot:run</code> (Linux/macOS: <code>./mvnw spring-boot:run</code>).
Frontend	<code>cd frontend</code> , <code>npm ci</code> if needed, then <code>npm run dev</code> . Open the Vite URL printed in the terminal.
API proxy	Vite proxies <code>/api</code> to <code>localhost:8080</code> . CSV sample downloads should be tested from the built backend-served app; the development proxy only covers <code>/api</code> .

Build implementation

`npm run build` performs strict TypeScript checking, then bundles React/TypeScript and CSS with esbuild. It writes `dist/index.html` and `assets/app.js + app.css`. Vite provides the development server; it is not the production bundler in this repository.

For Linux/macOS: build frontend, copy `dist/.` to `src/main/resources/static/`, run `./mvnw verify` and start the target JAR with `REVIEWER_TOKEN` set. Windows launcher execution has not been tested in the Linux build workspace.

Cloud deployment and updates

Render builds Docker from GitHub; Supabase stores persistent demo records.

Initial deployment

- Create a dedicated Supabase demo project and run supabase/setup.sql. Copy the Session pooler connection details.
- Create a Render Docker Web Service from the repository/main branch, or use render.yaml as a Blueprint. The demonstrated deployment uses Singapore.
- Set database variables and DB_INIT_MODE=never. Generate REVIEWER_TOKEN. Add Gemini key/model only when enabling provider calls.
- Use /api/health as the configured health path. Inspect startup logs and verify /api/accounts and /api/meta as well.
- Open the actual Render service URL, exercise all three fixtures, import the sample and save a review note.

Docker build stages

Stage	Implementation
Frontend	node:22-alpine; npm ci; npm run build.
Backend	maven:3.9.9-eclipse-temurin-17; copy built UI to static/; mvn -B verify.
Runtime	eclipse-temurin:17-jre-jammy; non-root app user; executable JAR; JAVA_TOOL_OPTIONS=-Xms64m -Xmx256m.

Update an existing checkout

```
git status
git add frontend/src/App.tsx frontend/src/styles.css
git commit -m "Update the review dashboard"
git push
```

Render redeploys when Git auto-deploy is enabled; otherwise trigger a manual deploy of the latest commit. Check the commit ID, wait for Live, then refresh the browser with Ctrl+F5. GitHub Actions also runs the frontend build and Maven verify on pushes and pull requests.

Cloud data lives in Supabase. An H2 file on the hosted container is not the intended durable store and may disappear on a restart/redeploy. Secrets belong in host settings, not GitHub. Free-service cold starts can delay the first request; inspect logs before treating a timeout as a code defect.

Verification and acceptance

Separate automated evidence, browser checks and live configuration demonstrations.

Automated Java suite	Tests	What it exercises
RiskEngineTest	13	Seed scores, time boundaries, other accounts, same timestamps, rapid ordering and one contribution per rule.
ApiTest	10	Public reads, unknown account, no-key label, token gating, review validation/history and metadata.
CsvImportServiceTest	10	Encoding/BOM, Thai text, quoting, timestamps, amounts, duplicate IDs, headers and row limits.
CsvImportApiTest	4	Unauthorized/invalid import creates nothing; fictional flag; independent cutoff and report inclusion.
ExplanationServiceTest	2	Simulated valid and invalid provider responses; fallback and no secret/error-body leakage.

Total: 39 tests in the recorded MVP verification. This documentation pass does not claim a new Maven test run: no Java code changed in the UI update. Strict TypeScript checking and the production frontend build were rerun for the redesign.

Redesigned browser checks

The updated built frontend was exercised in Chromium against the existing Java backend. Checks passed for evidence links, complete history, account switching, rule explanation, token entry, review saving/history, policy, automation, search, priority filters and CSV import/rejection. No browser runtime errors were recorded.

Overview widths 320, 390, 768, 1024, 1280, 1440 and 1920 pixels were checked for page and tab-bar horizontal overflow. At 1440 x 1000, the Save review action was within the initial viewport. Desktop/mobile dashboard and import views were rendered and visually inspected.

Live demonstration evidence

Owner confirmed deployed UI; Gemini output displayed "AI generated"; fictional sample import produced score 80; a saved pending-review note was independently retrieved through the account API. Cloud configuration works in the demonstrated flow. A controlled restart durability test and hosted n8n execution remain separate checks.

Rerun locally

```
./mvnw.cmd test
cd frontend
npm ci
npm run build
```

Troubleshooting

Start with the failing layer: editor, build, server, database or provider.

Symptom	Checks and action
Many JSX red lines in VS Code	If JSX.IntrinsicElements or React types are missing, run npm ci in frontend. Then TypeScript: Restart TS Server. Open the complete project rather than an isolated file. Run npm run build to verify actual compilation.
npm is not recognized	Confirm Node.js is installed and on PATH; reopen the terminal, then node --version and npm --version.
Hadoop Java works but this app fails	Check java -version; this app needs Java 17+. Choose a separate suitable JDK without breaking Hadoop's setup.
Render says Live, accounts fail	/api/health is liveness only. Inspect database/startup logs and request /api/accounts. Verify DB_URL, username, password and completed schema setup.
PostgreSQL connection / Broken pipe	Check Session pooler host/port, jdbc URL prefix, sslmode=require and credentials. The initial cloud session showed a transient failure then recovered; a definitive root cause was not established.
Save/import button disabled	Supply reviewer token; confirm server reviewerConfigured. Review needs a note; import needs file, name and fictional-data confirmation.
401 or 503 on a protected request	401: token absent/wrong. 503: server token unset. Re-enter access after reload; token is memory-only.
Rule fallback instead of AI generated	Check key/model, account availability, provider quota, timeouts and the six-attempt limiter. Fallback is intentional; it does not prove a successful provider call.
CSV rejected	Read the issues array. Check exact headers, UTF-8, quotes, timezone, positive decimal amount, unique IDs and limits. A valid-looking subset will not be saved from a bad batch.
Imported or reviewed data lost	Ensure the host uses Supabase rather than an ephemeral H2 file; verify connection settings and database state.
UI looks unchanged after a push	Check the deployed commit and build completion, then Ctrl+F5. A push and a finished deployment are distinct events.

Read server logs privately when investigating credentials or provider failures. The API intentionally hides raw provider error bodies; do not add keys or database passwords to public screenshots or issue reports.

n8n daily digest

Included as an inactive workflow export; activation is a separate deployment step.

Workflow topology

Daily schedule → Fetch risk report → Prepare digest.

The export defaults to 09:00 in Asia/Bangkok and active=false.

Configure it

- Import automation/n8n-daily-digest.json into your n8n instance.
- Change the request URL to <https://transaction-risk-review.onrender.com/api/reports/daily>, or your own deployed URL.
- Create an HTTP Header Auth credential: header name X-Reviewer-Token, value your private reviewer token. Assign it to Fetch risk report.
- Run manually. Confirm totalAccounts, highPriority, each account's reviewAsOf and linked evidence.
- Add an explicit destination if you want stored reports or notifications. Activate the schedule only after the manual run succeeds.

Digest shape (abridged)

```
{
  "snapshot": "2026-10-05T12:00:00Z",
  "dataMode": "SYNTHETIC", "totalAccounts": 3,
  "highPriority": [{
    "accountId": "DEMO-003",
    "reviewAsOf": "2026-10-05T12:00:00Z",
    "score": 80, "reviewStatus": "PENDING_REVIEW",
    "rules": [{ "title": "Transfer burst",
      "evidence": ["S-102", "S-103", "..."] }]
  }],
  "scope": "Fictional account snapshots..."
}
```

Implemented vs activated

The request node has a 90-second timeout. The Code node selects HIGH accounts and maps rule titles/evidence into a digest; its transformation was tested against a real Java report response. The export has no notification, storage or LLM node and contains no secret values.

The daily API top-level asOf is the seeded configuration cutoff. Imported accounts can have different cutoffs; use each analysis.asOf / reviewAsOf. A daily schedule does not turn fictional snapshots into a live banking feed.

Limitations and next increments

Keep claims proportional to what was built and demonstrated.

Current limits

- **Rules and data:** five hand-designed patterns, incomplete supplied histories and potentially overlapping evidence. No representative labelled dataset or measured fraud-classification performance.
- **Review governance:** shared token, no reviewer identity, case assignment, tenant isolation or tamper-evident audit mechanism.
- **AI:** JSON/length checks do not establish factual correctness. No multi-model failover, semantic validator, persistent explanation history or account-level provider quota.
- **Operations:** liveness-only health path, small connection pool and synchronous JDBC/provider requests. Public API exposure is appropriate only for fictional data in this demo.
- **Scaling:** account listing recomputes analysis and loads per-account data. Burst scanning is simple and can be quadratic in event count; no large-data benchmark or pagination is implemented.
- **Imports:** one file creates one account; no append, cross-file deduplication, delete, raw-file archive or bulk production ingestion.

Recommended development order

Priority	Increment	Evidence to collect
1	Durability and readiness	Save an import/review, restart the app and confirm persistence; add a DB-aware readiness path.
2	Hosted automation	Manual n8n run, scheduled execution and a chosen report destination.
3	Identity and audit	Individual authentication, permissions, actor attribution and case ownership.
4	Policy and data evaluation	More boundary scenarios, incomplete-history examples and measured reviewer workload on suitable labelled data.
5	UX and scaling	Two-decimal amount display, unsaved-note protection, pagination, efficient queries and performance tests.

Future work above is a roadmap, not part of the current implemented feature set. The project demonstrates integration and review workflow engineering; it does not certify compliance with any bank policy or privacy/security standard.

Demo and source guide

A short narrative you can explain without memorizing every line of code.

Three-minute walkthrough

- **Opening:** "I built a Java and TypeScript transaction-review prototype. Deterministic rules link signals to evidence; Gemini helps explain them; a human records the decision."
- **Show contrast:** DEMO-001 scores 0; DEMO-002 scores 35 despite a plausible supplier purchase; DEMO-003 scores 80 from four patterns. Explain why context matters.
- **Inspect evidence:** open the five burst transactions, then show full history and the earlier activity gap.
- **Explain and record:** generate AI analysis, identify its source label and save a fictional review note. Open Review history.
- **Show engineering:** import the sample, explain its independent cutoff and demonstrate that an invalid batch creates no account.

Interview questions and short answers

Question	Answer
Why Java rules plus an LLM?	Rules determine reproducible scores and evidence. The LLM produces readable context and checks, without deciding or changing the score.
Why BigDecimal?	Decimal amounts and threshold boundaries need predictable arithmetic rather than binary floating-point approximations.
How is import integrity maintained?	Validate the complete CSV first; save account, transactions and import metadata in one JDBC transaction.
How do you handle provider failure?	Return an explicit FALLBACK result; do not fake a successful Gemini response or expose provider internals.
What would you improve first?	Database-aware readiness and controlled restart verification, then identities/audit attribution and deployed automation.

Source and documentation maintenance

Primary implementation: github.com/pt22-mfu/transaction-risk-review. Frontend baseline: commit 3a1a8f2; application v0.2.0. Build/configuration: Dockerfile, render.yaml, pom.xml, frontend/package.json and .github/workflows/ci.yml. Existing notes: docs/PROJECT.md, docs/VERIFICATION.md and docs/DEMO_GUIDE.md.

Further reading: [Spring Boot](#) · [Supabase connections](#) · [Render Docker](#) · [Gemini structured output](#). These links are references, not a substitute for this project's actual code.

Document v1.0, 06 October 2026. Update this handbook when rules, API contracts, schemas or deployment settings change. Preserve the distinction between implemented, demonstrated and planned work.